DHBW Ravensburg

Fortgeschrittene Informatik

Prof. Dr. Andreas Schilling

www.ravensburg.dhbw.de

1. Spring Grundlagen

Agenda

Spring Boot

Beschreibung der Kernelemente von Spring Boot.

Dependency Injection

Kennenlernen und Einsetzten der Dependency Injection.

Spring Boot

Grundlagen

- Spring ist ein **offenes Framework** zur Erstellung von plattformunabhängigen Web-Anwendungen mit Java
- Die Kernprinzipien von Spring sind:
  - **Convention-over-Configuration**: Best-Practice Annahmen.
  - **Modularisierung**: Kleine erweiterbare Kernfunktionen.
  - **Schichtenorientierung**: Klare Strukturierung von Funktionalität.
  - **Dependency Injection**: Objekte bekommen notwendige Ressourcen zugewiesen.
- **Spring Boot** nutzt das Spring Framework inklusive Standard-einstellungen zur erleichterten Entwicklung von ausführbaren, self-contained, Webanwendungen in Form von JAR-Dateien.

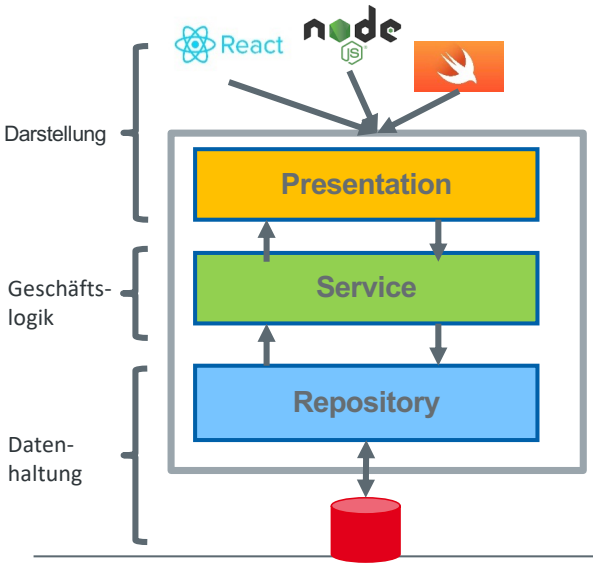
Spring ist ein sehr populäres Open-Source Framework zur Erstellung von Web-Anwendungen mit der Java Programmiersprache. Spring basiert auf vier Kernprinzipien:

- Convention-over-Configuration: Anstatt umfassender Konfiguration, werden Best-Practice Annahmen zugrunde gelegt und der Entwickler muss nur die Abweichungen zur diesen Best-Practice Annahmen spezifizieren.
- Hohe Modularisierung: Statt einer umfassenden Bibliothek ist das Spring Framework in verschiedene Module untergliedert. Diese hohe Modularisierung dient dazu, dass nur jene Bibliotheken geladen werden, die auch tatsächlich benötigt werden.
- Schichtenorientierung: Das Spring Framework strukturiert eine Java-Anwendung in die Funktionsschichten: Präsentation, Geschäftslogik und Datenhaltung.
- Dependency Injection: Um die Komplexität zu reduzieren übernimmt Spring zum Teil die Verwaltung von Objekten.

Spring Boot setzt auf dem Spring Framework auf und ergänzt dieses um Standardeinstellungen sodass die Entwicklung von Webanwendungen stark vereinfacht wird. Darüber hinaus ermöglicht es ausführbare Webanwendungen incl. Webserver in Form von JAR-Dateien zu erstellen. Dies war zuvor undenkbar, da für die Ausführung von Webanwendungen vor Spring Boot stets ein dedizierter Anwendungsserver (bspw. Tomcat) benötigt wurde.

Spring Boot

Schichtenmodell



Present. In dieser Schicht werden Objekte gesammelt welche die Darstellung der Ein- & Ausgaben der Client Aufrufe betreffen.

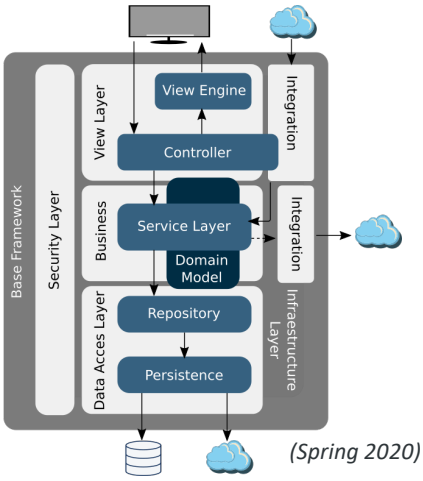
Business Diese Schicht kapselt alle Objekte, welche die reine Geschäftsfunktionalität umsetzen.

Data In dieser Schicht werden alle Objekte zusammengefasst, welche die Ansprache und Übersetzung der Datenspeicherung betreffen.

Das Spring Framework strukturiert die Umsetzung einer Web-Anwendung in drei grundlegende Schichten. Die erste Schicht widmet sich der Präsentation der Benutzeroberfläche, sowie um die Ein- und Ausgabe von Daten. Die zweite Schicht widmet sich der Umsetzung der Geschäftsfunktionalität, indem sie die Geschäftslogik samt der verwendeten Geschäftsobjekte umfasst. Abschließend befasst sich die Datenschicht mit der Speicherung und der Ansprache der Datenobjekte.

Spring Boot

Schichtenmodell II



(Spring 2020)

Spring baut auf dem Entwurfsmuster der Schichtentrennung auf und unterscheidet folgende wichtige Objektklassen:

Controller

Diese Kategorie kennzeichnet alle Klassen, welche Client-Anfragen annehmen und die Antworten darauf in verschiedenen Formaten zurückgeben.

Service

Diese Objektklasse umfasst alle Klassen, welche die Geschäftslogik umsetzen.

Repository

Unter dieser Kategorie werden alle Datenobjekte klassifiziert, die in einer Datenbank gespeichert werden.

Basierend auf der zuvor vorgestellten Schichtentrennung unterscheidet das Spring Framework drei wichtige Objektklassen:

- Bei der Controller Kategorie handelt es sich um Objekte, welche sich der Entgegennahme und der Beantwortung von Benutzeranfragen widmen. Dies kann zum einen in Form eines Application Programming Interface (API) gestaltet sein, bei dem der Benutzer seine Anfragen an eine Web-Schnittstelle stellt, oder in Form einer grafischen Benutzeroberfläche.

- Unter der Service Kategorie werden jene Objekte gruppiert, welche die Geschäftslogik umsetzen und hierzu auch Services von Drittanbietern ansprechen.

- Die Repository Kategorie gruppiert jene Datenobjekte die gespeichert werden und übernimmt die Ansprache und Konfiguration der Datenbank.

Spring Boot

Konfiguration

- In Spring können **große Teile** der Konfiguration über **Annotationen** vorgenommen werden.
- Annotationen sind **deklarative Sprachelemente**, die mit ‚**@**' beginnen und an **Klassen, Attribute, Methoden** und **Parameter** geschrieben werden.
- Annotationen werden eingesetzt für:
  - Anmerkungen (bspw. @Deprecated)
  - Konfigurationen (bspw. @Entity)
  - Spracherweiterungen (bspw. @Slf4j, @Getter)

```
@Getter
@Setter
@NoArgsConstructor
@Entity
public class Movie {
```

Annotationen an einer Klasse können Funktionen erweitern.

Spring zeichnet sich gegenüber anderen Frameworks dadurch aus, dass es für die Konfiguration stark auf Java Annotationen setzt. Der grundlegende Gedanke dahinter ist, dass die Konfiguration direkt im Code als Anmerkung vorgenommen wird. Hierdurch ist die Konfiguration für den Benutzer stets nachvollziehbar und in direkter Nähe zum betreffenden Quelltext.

Annotationen sind ein zentraler Bestandteil der Java Programmiersprache und beginnen mit einem ‚@' Zeichen gefolgt vom jeweiligen Befehl. Annotationen können an Klassen, Attribute, Methoden und Parameter geschrieben werden.

In Java werden Annotationen eingesetzt um verschiedene Konfigurationen vorzunehmen. Eine Einsatzform betrifft die Dokumentation. Hier kann die Annotation @Deprecated eingesetzt werden um Entwickler auf die Verwendung veralteter Methoden aufmerksam zu machen. Darüber hinaus können Annotationen eingesetzt werden um konkrete Konfigurationen vorzunehmen. Wie wir später ausführlich sehen werden dient die @Entity Annotation dazu Objekte in der Datenbank zu speichern. Abschließend können Annotationen dazu eingesetzt werden um Funktionalität nachzurüsten welche standardmäßig nicht in der Java Programmiersprache vorgesehen ist. Beispielsweise wird mit der Ergänzung @Slf4j bei einer Klasse automatisch ein Logger erzeugt. Dieser Logger kann innerhalb der Klasse direkt verwendet werden, bspw. durch den Aufruf log.info(„Hello World!")

Spring Boot

Beans

- Eine **Bean** entspricht einer Java Klasse deren Lebenszyklus (Erstellung, Konfiguration, Skalierung, Löschung) durch Spring verwaltet wird.
- Durch die Verwaltung der Beans durch Spring, müssen diese **nicht manuell via „new“** instanziiert werden.
- Beans werden **wie normale Java-Klassen** spezifiziert und durch eine **Annotation als Bean deklariert,** sowie einer Funktions-schicht (bspw. Präsentation) zugewiesen.
- Die Zuweisung einer Bean zur einer Funktionsschicht wird als **Stereotyp** bezeichnet.

Spring Beans sind eine Kernkomponente des Spring Frameworks. Unter Beans versteht man Java Klassen deren Lebenszyklus vom Spring Framework verwaltet wird. Das heißt das Spring Framework ist verantwortlich für die Instanziierung der Klasse, deren Konfiguration, Skalierung, sowie deren Löschung, sofern sie nicht mehr benötigt wird. Ganz konkret bedeutet dies, dass Beans nicht wie traditionelle Java Klassen manuell via ‚new‘ instanziiert werden, sondern im Hintergrund durch das Spring Framework instanziiert werden.

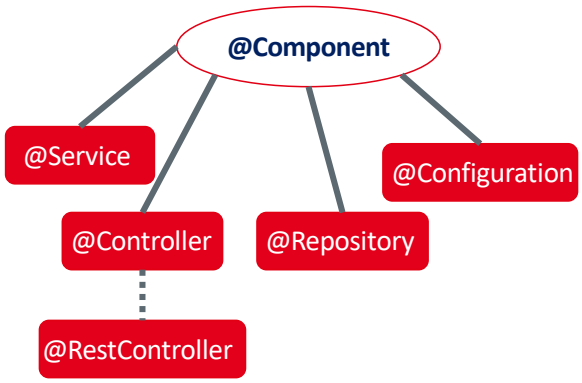
Spring Beans werden genauso wie normale Java-Objekte erstellt. Im Gegensatz zu traditionellen Java Objekten besitzen die Spring Beans eine Annotation die eine Zuordnung zu einer Funktionsschicht (Controller, Service, Repository) vornimmt. Diese Zuordnung wird als Stereotyp bezeichnet.

Spring Boot

Beans - Stereotype

Folgende Stereotypes werden unterschieden:

- ***@Component*** (Kernkomponente) ernennt eine Klasse zur Bean deren Lebenszyklus von Spring verwaltet wird.
- ***@Controller*** klassifiziert Beans, die zur Kommunikation mit dem Client verwendet werden. Die Subklasse @RestContoller gibt den Output in JSON Notation aus.
- ***@Service*** klassifiziert Beans, die zur Erbringung von Geschäftsfunktionalität eingesetzt werden.
- ***@Repository*** beschreibt Beans welche zur Speicherung von Daten eingesetzt werden.
- ***@Configuration*** dient zur Infrastrukturkonfiguration („Configuration as Code“).



Das Spring Framework unterscheidet folgende Stereotypes (Funktionszuordnungen) für Beans:

- Die **@Component** Annotation ist die Oberklasse aller Stereotypen. Diese Annotation kennzeichnet Klassen deren Lebenszyklus durch Spring verwaltet wird. Es wird jedoch keine explizite Einteilung in eine Funktionsschicht vorgenommen.
- Mit der Annotation **@Controller** werden Beans klassifiziert welche der Kommunikation mit dem Benutzer dienen. Zum Anbieten einer REST-API (vgl. Kapitel 3) bietet sich die Verwendung der Annotation **@RestController** an. Diese Annotation bewirkt, dass sämtlicher Output der Methoden im JSON-Format zurückgegeben wird.
- Beans die der Umsetzung der Geschäftslogik dienen werden mit der Annotation **@Service** ausgezeichnet
- Die Annotation **@Repository** kennzeichnet jene Beans die zur Speicherung der Objekte in der Datenbank eingesetzt werden.
- Abschließend ermöglicht die Annotation **@Configuration** Konfigurationen am Spring Framework in Form von Code vorzunehmen, hierzu kommt das Paradigma Configuration-as-Code zum Tragen welches vorsieht, dass Konfigurationen nicht in Dateiform, sondern in Programmcode ausgedrückt werden.

Agenda

Spring Boot

Beschreibung der Kernelemente von Spring Boot.

Dependency Injection

Kennenlernen und Einsetzten der Dependency Injection.

Dependency Injection

Inversion of Control (IoC)

- **Inversion of Control (IoC)** ist ein zentrales Paradigma in der Softwareentwicklung das vorsieht, dass Objekte so stark wie möglich voneinander **entkoppelt** sind.
- Der Grundgedanke bei IoC ist, dass die Absicht der Verwendung eines Objekts höher priorisiert wird als die Kenntnis der Implementierungs-details dieses Objekts.

```
public class Car{
    private PetrolEngine petrolengine;
    public Car(){
        this.petrolengine = new PetrolEngine(6);
    }
}
```

Zur manuellen Instanziierung sind Details der Implementierung notwendig.

```
public interface Engine() {...}

public class Car{
    private Engine engine;
    public setEngine(Engine engine){
        this.engine = engine;
    }
}

Car newCar = new Car()
Engine petrolEngine = new PetrolEngine(6);
newCar.setEngine(petrolEngine);
```

- Eine Möglichkeit IoC umzusetzen ist die **Verwendung von Interfaces samt Getter & Setter** Methoden. Hierbei ist es unumgänglich einen Bereich im Code zu haben, der die gewünschte Konfiguration vornimmt.

Inversion of Control (IoC) bezeichnet ein zentrales Paradigma in der Softwareentwicklung gemäß dessen Objekte, die miteinander interagieren, so stark wie möglich voneinander entkoppelt werden. Nehmen wir zur Veranschaulichung dieses Prinzips folgende Ausgangssituation: Bei der Java-Klasse „Car“ wird eine Instanz der Klasse „PetrolEngine“ benötigt. Eine enge Kopplung zwischen diesen beiden Klassen liegt vor, wenn die Klasse Car, die Instanziierung der Klasse PetrolEngine selbst vornimmt, da sie hierzu Kenntnisse über die Interna der Klasse PetrolEngine wie beispielsweise die Zylinderzahl kennen muss. In Folge dessen führt eine nachträgliche Änderung an der Klasse „PetrolEngine“ auch zu Anpssungen an der Klasse „Car“

Der Grundgedanke von IoC besagt, dass die Verwendung einer Klasse ohne Kenntnis über deren Implementierungsdetails möglich sein soll. Eine Form dieses Prinzip umzusetzen ist die konsequente Verwendung von Interfaces und Getter & Setter Methoden. Angewandt auf unser Ausgangsbeispiel bedeutet dies, dass die Klasse Car nicht mehr die Klasse PetrolEngine referenziert sondern deren Interface ‚Engine‘. Ferner wird die Instanziierung der PetrolEngine Klasse außerhalb der Car-Klasse vorgenommen und via Setter Methode dem Car-Objekt zugewiesen. Durch diese Abstraktion und der ausgelagerten Konfiguration ist es möglich, die Implementierung der Engine Klasse komplett auszutauschen, ohne dass Änderungen an der Klasse „Car“ notwendig sind.

Dependency Injection

Überblick

- Dependency Injection ist eine komfortable Möglichkeit das **IoC-Paradigma** umzusetzen. Hierbei verwaltet und **instanziiert die Laufzeitumgebung die Abhängigkeiten** einer Klasse.
- Wie bisher werden die Klassen via Interfaces entkoppelt, doch der **statische Konfigurationscode entfällt**.
- In Spring werden jene Komponenten von denen eine Bean abhängt mit **@Autowired** überschrieben. Im Anschluss daran kümmert sich Spring um die Auswahl einer geeigneten Implementierung.
- **Wichtige Vorteile** die durch Dependency Injection entstehen:
  - Getrennte Weiterentwicklung
  - Reduzierung von Codekomplexität (vgl. Kapitel 2 & 3)
  - Verbesserte Testmöglichkeiten (vgl. Kapitel 4)

```
14  public class Car{
15      @Autowired
16      private Engine engine;
17  }
18
19  @Component
20  public class InlineSixCylinderEngine implements Engine{
21    //...
22  }
```

Im Gegensatz zur Setter-Konfiguration übernimmt Spring die Instanziierung und Zuweisung der Klassen.

Dependency Injection stellt eine weitere Möglichkeit dar das IoC Paradigma umzusetzen. Im Gegensatz zur Verwendung der Getter & Setter Methoden wird hier die Konfiguration und Instanziierung der Objekte durch die Laufzeitumgebung (also Spring) umgesetzt. In Folge dessen entfällt die statische Konfiguration der Klassen.

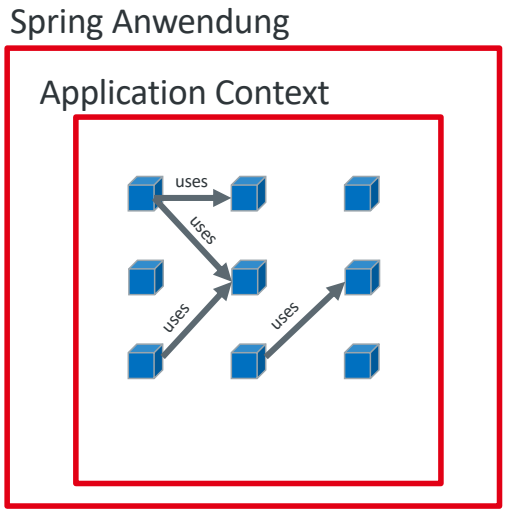
Zur Umsetzung von Dependency Injection werden jene Komponenten von denen eine Bean abhängt mit @Autowired annotiert. Spring sucht dann für diese Komponenten nach einer geeigneten Implementierung und instanziiert diese.

Durch die Verwendung von Dependency Injection lassen sich wichtige Vorteile bei der Softwareentwicklung umsetzten. Einer der Vorteil betrifft die getrennte Weiterentwicklung von Klassen, die miteinander interagieren, da durch DI das notwendige Wissen für die Interaktion maßgeblich reduziert wird. Darüber hinaus bewirkt DI eine Reduktion der Codekomplexität da die Konfiguration der interagierenden Objekte an die Laufzeitumgebung ausgelagert wird. Abschließend verbessert DI die Testmöglichkeiten von Web-Anwendungen, da durch die lose Kopplung die verwendeten Komponenten, ohne den jeweiligen Code zu ändern, gegen Test-Facaden ausgetauscht werden können.

Dependency Injection

Component Scan

- Beim Applikationsstart einer Spring Anwendung wird der **Component Scan** durchgeführt.
- Im Rahmen des **Component Scan** indiziert und scannt Spring alle Klassen die mit **@Component** oder Subklassen davon annotiert sind.
- Des Weiteren wird die **Struktur** der identifizierten Beans gescannt und **benötigte Abhängigkeiten instanziiert**.



Beim Start einer Spring Anwendung wird der sogenannte Component Scan durchgeführt, der das Ziel hat alle Klassen zu identifizieren, die mit @Component oder Subtypen hiervon ausgezeichnet sind (bspw. @Controller oder @Service). Neben der Identifikation der entsprechenden Klassen wird ihre Struktur gescannt und benötigte Abhängigkeiten werden instanziiert.

Dependency Injection

Umsetzungsarten

- **Constructor Injection**: @Autowired über dem Konstruktor

  Spring fügt die Abhängigkeiten über Konstruktorargumente ein. Seit Spring Version 3.5 ist dies eine **optionale Annotation**. Die Constructor Injection gilt als **Best-Practice.**

```java
public class MovieRecommender {

    private final CustomerPreferenceDao customerPreferenceDao;

    @Autowired
    public MovieRecommender(CustomerPreferenceDao customerPreferenceDao) {
        this.customerPreferenceDao = customerPreferenceDao;
    }
```

- **Setter Injection**: @Autowired über dem Setter

  Die Abhängigkeiten werden **nach Instanziierung** der Klasse im Rahmen der Initialisierung eingefügt.

```java
public class SimpleMovieLister {

    private MovieFinder movieFinder;

    @Autowired
    public void setMovieFinder(MovieFinder movieFinder) {
        this.movieFinder = movieFinder;
    }
```

- **Field Injection**: @Autowired über dem Klassenattribut

  Die Abhängigkeiten werden **nach Instanziierung** und Initialisierung über Reflection eingefügt.

```java
public class MovieRecommender {

    @Autowired
    private MovieCatalog[] movieCatalogs;
```


Die Nutzung von Dependency Injection kann an verschiedenen Stellen im Code geschehen:

- Zum einen kann eine Klasse die Objekte, die für ihren Funktionsumfang notwendig sind, als Argumente im Konstruktor referenzieren. In diesem Fall wird @Autowired über dem Konstruktor platziert. Im Rahmen des Component Scans sucht Spring nach einer geeigneten Implementierung für die Konstruktorargumente und instanziiert diese. Seit Version 3.5 des Spring Frameworks ist die Verwendung von @Autowired bei einer konstruktorbasierten DI optional. Die Constructor Injection gilt als Best-Practice.
- Eine weitere Möglichkeit DI in Spring zu verwenden ist die @Autowired Annotation über einer Setter-Methode. Bei dieser Form der Dependency Injection werden die Abhängigkeiten nach der Instanziierung gesetzt
- Die letzte Möglichkeit der Verwendung von Dependency Injection besteht in der Field Injection bei der @Autowired über dem entsprechenden Klassenattribute platziert wird. Hierbei werden die Abhängigkeiten ebenfalls erst nach der Instanziierung der Klasse gesetzt.

Aufgabe 1

Dependency Injection

a) Erweitern Sie die Klasse **MovieControllerImpl** um eine weitere Methode **getMovieServiceName**(), welche die **getName**() Methode von **MovieServiceImpl** aufruft und das Ergebnis zurückgibt. Testen Sie Ihne Implementierung in der **ManualTestBean**.

b) Setzen Sie den **Aufruf** von **MovieControllerImpl** in der **ManualTestBean** und **MovieServiceImpl** in **MovieControllerImpl** via Interfaces um.

c) Erstellen Sie den Service **ArtistServiceImpl** analog zu MovieServiceImpl mit der Methode **getName**() und sprechen Sie den neuen Service samt der neuen Methode vom **MovieControllerImpl** aus an. Testen Sie Ihre Umsetzung mit der **ManualTestBean.**